mtl: https://hackage.haskell.org/package/mtl-2.3
transformers: https://hackage.haskell.org/package/transformers-0.6.0.4

```
class Monad m => MonadState s m | m -> s where
    get :: m s
    put :: s -> m ()
    state :: (s -> (a, s)) -> m a
    {-# MINIMAL state | get, put #-}

modify :: MonadState s m => (s -> s) -> m ()
gets :: MonadState s m => (s -> a) -> m a

class Monad m => MonadReader r m | m -> r where
    {-# MINIMAL (ask | reader), local #-}
    ask   :: m r
    local :: (r -> r) -> m a -> m a
    reader :: m (r -> a) -> m a
asks :: MonadReader r m => (r -> a) -> m a

class (Monoid w, Monad m) => MonadWriter w m | m -> w
where
    {-# MINIMAL (writer | tell), listen, pass #-}
    writer :: (a,w) -> m a
    tell   :: w -> m ()
    listen :: m a -> m (a, w)
    pass   :: m (a, w -> w) -> m a

listens :: MonadWriter w m => (w -> b) -> m a -> m (a,
b)
censor :: MonadWriter w m => (w -> w) -> m a -> m a

class (Monoid w, MonadReader r m, MonadWriter w m,
MonadState s m)
   => MonadRWS r w s m | m -> r, m -> w, m -> s
```

Instances

```
-- simple instances
instance MonadReader r (r->)
instance MonadWriter w (w,)

-- Transformers
instance Monad m => MonadState s (StateT s m)
instance MonadState s m => MonadState s (Identity e m)
instance MonadState s m => MonadState s (ExceptT e m)
instance MonadState s m => MonadState s (MaybeT m)
instance MonadState s m => MonadState s (ReaderT r m)
instance (Monoid w, MonadState s m) => MonadState s
(WriterT w m)
instance (Monoid w, Monad m) => MonadState s (RWST r w s
m)
instance MonadState s m => MonadState s (ContT r m)
```

```
instance MonadState s m => MonadState s (Select r m)
...
```

Example: classic examples
Classic example of **MonadReader** is a **function (r ->)**
Classic example of **MonadWriter** is a **tuple (w,)** with **Monoid w**

```
-- MonadReader Int (Int->)
--
f :: Int -> String
f = do
  y <- local (* 2) (show . (+1) <&> ask)       --
local :: (Int -> Int) -> (Int -> String) -> (Int ->
String)
                                                -- y =
"21"    (x*2) + 1 where x = 10
  x <- ask                                      -- ask ::
Int -> Int
                                                -- x = 10
  return $ "value:" <> show (x + read y)
f 10                      -- "value:30"


-- MonadWriter String (String,)
-- Note: Monoid String
--
t :: (String, Int) = do
  tell "abc"                        -- tell :: String ->
(String,Int)
  tell "def"
  writer (10, "123")                -- writer :: (Int,
String) -> (String,Int)

t                         -- ("abcdef123", 10)
```

Usage: transformer composition
One of the primary usages of transformer classes is **composition**.

e.g. compose ReaderT + WriterT + StateT + ExceptT + IO

```
-- MyApp0 = (MonadReader, MonadWriter, MonadState,
MonadError, MonadIO)
type MyApp0 r w s e = ReaderT r (WriterT [w] (StateT s
(ExceptT e IO)))

-- with transformer class
taskWithTransformLift :: MyApp0 Bool Int String String
Bool
taskWithTransformLift = do
  x <- ask                      -- MonadReader
  tell [0]                      -- MonadWriter
```

```
  put "Hello"                    -- MonadState
  _ <- throwError "Oops"         -- MonadError
  liftIO $ putStrLn "Hello"      -- MonadIO
  return x


-- e.g. without transformer class
-- i.e. manual lifting
taskWithManualLift :: MyApp0 Bool Int String String Bool
taskWithManualLift = do
  x <- ask                                    --
level1: ReaderT
  lift $ tell [0]                             --
level2: WriterT
  lift $ lift $ put "Hello"                   --
level3: StateT
  _ <- lift $ lift $ lift $ throwError "Oops"   --
level4: ExceptT
  lift $ lift $ lift $ lift $ putStr "Hello"    --
level5: IO
  return x
```

usage: derived instances

Transformer classes can usually be **derived** using the **Generalized derived instances** GHC extension.

GeneralizedNewTypeDeriving: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/newtype_deriving.html

e.g.

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Control.Monad.Except
import Control.Monad.Reader
import Control.Monad.State
import Control.Monad.Writer

newtype MyError = MyError String
newtype MyApp1 r w s a = MyApp1 {runMyApp :: ReaderT r
(WriterT [w] (StateT s (ExceptT MyError IO))) a}
  deriving
    ( Functor,
      Applicative,
      Monad,
      MonadReader r,
      MonadState s,
      MonadWriter [w],
      MonadError MyError,
      MonadIO
    )

processMyApp :: MyApp1 r w s a -> r -> s -> IO (a, [w],
s)
processMyApp (MyApp1 app) r st = do
```

```
  Right ((res, logs), st') <- runExceptT $ runStateT
(runWriterT (runReaderT app r)) st
  return (res, logs, st')

myTask :: MyApp1 Bool Int String Bool
myTask = do
  x <- ask
  tell [1]
  put "test"
  liftIO $ putStrLn "Hello"
  return x

main :: IO ()
main = processMyApp myTask True "" >>= print
```

Output

```
Hello


            (True,[1],"test")
```

e.g. Same classes implemented manually

```
{-# LANGUAGE FlexibleInstances #-}                    --
e.g. need for `MonadState s (MyApp l s)` i.e. s
appearing in both variables
{-# LANGUAGE GeneralizedNewtypeDeriving #-}           --
need for deriving Functor, Applicative, Monad
{-# LANGUAGE MultiParamTypeClasses #-}                --
e.g. need for `MonadWriter [l] (MyApp l s) i.e. multiple
type parametrs

module MyApp where

import Control.Monad.Except
import Control.Monad.Reader
import Control.Monad.State
import Control.Monad.Writer

newtype MyError = MyError String
newtype MyApp2 r w s a = MyApp2 {runMyApp2 :: ReaderT r
(WriterT [w] (StateT s (ExceptT MyError IO))) a}
  deriving
    ( Functor,
      Applicative,
      Monad
    )

instance MonadWriter [w] (MyApp2 r w s) where
  writer = MyApp2 . lift . writer
  listen (MyApp2 r) = MyApp2 $ listen r
```

```
  pass (MyApp2 r) = MyApp2 $ pass r

instance MonadState s (MyApp2 r w s) where
  get = MyApp2 $ lift $ lift get
  put = MyApp2 . lift . lift . put

instance MonadReader r (MyApp2 r w s) where
  ask = MyApp2 ask
  local f (MyApp2 app) = MyApp2 (local f app)

instance MonadIO (MyApp2 r w s) where
  liftIO io = MyApp2 (lift $ lift $ lift $ lift io)

instance MonadError MyError (MyApp2 r w s) where
  throwError e = MyApp2 $ throwError e
  catchError (MyApp2 p) h = MyApp2 $ catchError p
(runMyApp2 . h)
```